

Research Proposal

CID: 02538725

1 Introduction

Dropout (Hinton et al. 2012) is the standard regularisation technique for deep neural networks: during training, each neuron in a given layer is independently zeroed with a fixed probability p . This serves as a regularisation technique preventing the co-adaptation of features that contribute to overfitting. Despite its popularity, vanilla dropout is *topology-blind*. It applies the same retention probability to every neuron in a given layer, regardless of its role in the network’s information flow. This proposal raises the question of whether a blanket probability for every neuron in a layer neglects to incorporate topological properties of the weight graph on which information flows through the network.

This project proposes **curvature-aware dropout**, a method that uses discrete Ricci curvature, computed on the weighted graph induced by a Multi-Layer Perceptron’s (MLP’s) connection weights, to assign non-uniform, topology-informed dropout probabilities to individual neurons. The core hypothesis is that neurons anchoring edges of highly negative curvature perform unique, non-redundant computation and should be retained more often; while neurons embedded in positively curved (redundant) neighbourhoods can be dropped more aggressively. The primary research question is: **does curvature-aware dropout improve generalisation in MLPs compared to uniform dropout, and if so, under what conditions?**

2 Literature Review

Discrete Ricci curvature for networks: Ollivier discretised Ricci curvature (Ollivier 2007) for metric spaces by comparing the Wasserstein-1 distance between neighbourhood distributions to the graph distance between their centres: for an edge $\{u, v\}$, the curvature is

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)}. \quad (1)$$

Positively curved edges connect nodes whose neighbourhoods overlap heavily (redundancy), while negatively curved edges bridge disparate regions (bottlenecks). Forman (2003) provided an alternative combinatorial discretisation via the Bochner–Weitzenböck formula. For an edge e connecting vertices v_1 and v_2 in a weighted network, the Forman–Ricci curvature is :

$$F(e) = w_e \left(\frac{w_{v_1}}{w_e} + \frac{w_{v_2}}{w_e} - \sum_{e' \sim e} \left(\frac{w_{v_1}}{\sqrt{w_e w_{e'}}} + \frac{w_{v_2}}{\sqrt{w_e w_{e'}}} \right) \right), \quad (2)$$

(Samal et al. 2018, Saucan et al. 2019) where $w_e, w_{e'}$ are edge weights, w_{v_i} are vertex weights, and $e' \sim e$ denotes edges adjacent to e (excluding e itself). In the unweighted case ($w_e = w_v = 1$), this simplifies to $F(e) = 4 - \deg(v_1) - \deg(v_2)$. Samal et al. (2018) conducted empirical comparison across model and real-world networks, finding that Forman–Ricci and Ollivier–Ricci curvature are highly correlated in many network types, with Forman–Ricci curvature being orders of magnitude cheaper to compute. Azarhooshang et al. (2020) however, reviewed efficient computation methods for Ollivier–Ricci curvature, including linear-sketching approximations that would allow the exploration of a comparison between the two curvature methods.

Direct Precedents: There are many recent applications of discrete Ricci curvature to neural networks. Perhaps the most relevant is the CurvDrop method (Liu, Zhou, Pan, Wu, Li, Chen & Zhang 2023) that directly integrates discrete Ricci curvature into a topology-aware edge dropout scheme for Graph Neural Networks, to simultaneously address over-smoothing and over-squashing. Edges with highly negative curvature (that form bottlenecks) receive lower probability, while positively curved (redundant) edges are dropped more frequently. This project, on the other hand, proposes to work directly on the *weight graph* of an MLP, rather than the input graph edges to a GNN. Extending this dropout framework to the MLP and non-graph neural networks is therefore a novel proposal of this research.

Motivating the usage of curvature methods to identify information bottlenecks in neural networks *in general* was recently addressed by Tan et al. (2026) in “Analyzing Neural Network Information Flow Using Differential Geometry” which applies Ollivier-Ricci curvature directly to the weight graph of a neural network. The authors found that edges with negative Ollivier-Ricci curvature are information bottlenecks that are essential to network connectivity, while edges with positive ORC are redundant and able to be pruned. These findings serve as empirical support for the core intuition behind a curvature aware dropout scheme that this research will investigate.

Adaptive Dropout: There are several authors who have investigated methods of intelligently assigning non-uniform dropout rates. With **Targeted Dropout** Gomez et al. (2018) explored assigning dropout probabilities based on the ‘importance’ of weights as implied by their magnitude. This reflects a similar principle to what this paper proposes, with weight magnitude being used instead of discrete curvature of the same edge. While others have intelligently set dropout rates per neuron based on other metrics, no one has examined using the interpretation of Ricci curvature in networks to do so.

Ricci curvature and network structure: One can also consider the work done on the interpretation of Ricci curvature in networks. Topping et al. (2022) showed that negatively curved edges are responsible for over-squashing in message-passing GNNs. Nguyen et al. (2023) provided a unified framework showing that positive curvature drives over-smoothing. Positive curvature exemplifies neighbourhoods converging (akin to a sphere in geometry) including redundant edges within well-connected clusters. Negative curvature is present when neighbourhoods diverge (akin to a saddle in geometry), indicating bottleneck edges that carry unique information that should be preserved. Importantly, it can be shown that discretized notions of Ricci curvature for edges in graphs can be extended to nodes (Weber et al. 2016), allowing us to assign curvature values to the neurons in a neural network.

3 Proposed Methodology

Weight graph construction: Given a trained or partially trained MLP with layers $L_0, \dots, L_K$, we construct a weighted graph $G = (V, E)$ for each adjacent layer pair (L_k, L_{k+1}) , where edges carry weights w_{ij} (absolute value of the learned parameter). Neuron i in layer k has degree equal to the width of layer $k+1$ and vice versa, so the graph is fully connected, but with non-uniform weights.

Curvature computation: The primary curvature measure will be **Forman–Ricci curvature**, chosen for computational feasibility: for an unweighted graph the per-edge cost is $O(\deg(u) + \deg(v))$, yielding total cost $O(|E| \cdot d_{\max})$ per layer pair. For a 784-512-256-10 MLP this is of order 10^8 , computable in seconds. The weighted generalisation from Samal et al. (2018), Saucan et al. (2019) (Eq. 2 in this proposal) will be used to incorporate edge weights. Ollivier–Ricci curvature will serve as a secondary comparison on smaller architectures, computed via the GRAPHRICCICURVATURE library (Ni et al. 2019).

Curvature-to-dropout mapping: Neuron-level curvature is defined by averaging the Ricci curvature over connected edges:

$$\kappa(v) = \frac{1}{\deg(v)} \sum_{\{u,v\} \in E} \kappa(u, v). \quad (3)$$

The neuron’s dropout probability is then set via a monotone mapping that assigns lower dropout to neurons with more negative curvature (bottlenecks) and higher dropout to neurons with positive curvature (redundant). A natural candidate is a shifted sigmoid:

$$\tilde{p}_i = p_{base} \cdot \sigma(\alpha(\kappa(v_i) - \beta)), \quad (4)$$

where p_{base} controls the overall dropout budget, α the sensitivity to curvature, and β an offset. These hyperparameters will be tuned via validation performance. Importantly, the mapping would then be re-normalised so the expected number of dropped neurons per layer equals that of uniform dropout at rate p_{base} , ensuring a fair comparison:

$$p_i = \frac{n \cdot p_{base}}{\sum_{j=1}^n \tilde{p}_j} \cdot \tilde{p}_i$$

Training protocol. Curvature is uninformative when weights are randomly initialised. Following the logic of Curriculum Dropout (Morerio et al. 2017) and the empirical finding that early-training dropout can reduce gradient variance (Liu, Xu, Jin, Shen & Darrell 2023), the proposed protocol is: (i) train with standard uniform dropout for an initial warm-up phase of N_{warm} epochs; (ii) compute curvature on the weight graph and activate curvature-aware dropout; (iii) periodically recompute curvature every Δ epochs as weights evolve. The warm-up length N_{warm} and recomputation interval Δ are treated as hyperparameters.

4 Experimental Design

Datasets and architectures: The datasets that will be used are MNIST as a sanity check, and CIFAR-10 as the primary benchmark. Both datasets are publicly available and pose no ethical concerns. MLP architectures of varying width and depth (e.g., 784-512-256-10 and 784-1024-512-256-10 for MNIST; 3072-1024-512-256-10 for CIFAR-10) will be used in experiments. All models will be implemented in Python 3 with PyTorch.

Baselines and comparisons: The primary comparison is against uniform dropout at matched dropout budgets. Additional baselines include: (a) Targeted Dropout (Gomez et al. 2018), which assigns non-uniform rates based on weight magnitude; (b) no dropout; (c) a *random-curvature control* where dropout rates are drawn randomly. This will be to verify that the curvature signal itself, rather than mere non-uniformity, is driving any improvement.

Evaluation metrics: The primary metric is the test-accuracy gap between curvature-aware and uniform dropout. Secondary metrics include: the train vs test accuracy of the curvature-aware dropout, the curvature distribution evolution across training, and some measure of the computational overhead of curvature re-computation.

Extensions: Key studies include: (a) Forman–Ricci vs. Ollivier–Ricci curvature (on a smaller network); (b) warm-up length N_{warm} ; (c) curvature re-computation frequency Δ ; (d) sensitivity parameter α ; (e) a *weight-magnitude control* where dropout probabilities are set proportional to the mean absolute weight per neuron rather than curvature, to test whether the geometric signal in Eq. 2 adds information beyond raw magnitude.

5 Feasibility, Limitations, and Timeline

The project is designed to be completable in 12 weeks (June 1 – August 28). Table 1 summarises the planned schedule. Writing will occur at virtually all stages of the project to ease the burden of writing the report documenting current progress.

Weeks	Phase	Tasks
1–2	Setup & baselines	Baseline MLP results on MNIST/CIFAR-10 with uniform dropout
3–4	Curvature pipeline	Weight-graph construction; Forman–Ricci validated on toy networks
5	Integration	Curvature-to-dropout mapping; first end-to-end training runs
6–8	Core experiments	Systematic comparison across architectures and baselines
9	Extended studies	Random-curvature study; degree-based study; ORC comparison
10–11	Analysis & writing	Final figures, statistical analysis, and report draft
12	Buffer & polish	Final revisions and submission

Table 1: Planned project timeline.

Potential limitations and mitigation: The principal risk is that curvature-aware dropout may undermine the stochastic regularisation properties of standard dropout by deterministically favouring certain neurons. A second concern is computational overhead; Forman–Ricci curvature is chosen precisely because it adds negligible cost relative to a training epoch. Finally, MLPs are fully connected, so the weight graph has regular structure that may limit curvature’s discriminative power. If curvature values cluster narrowly, the method reduces to near-uniform dropout—a negative but informative result. If this ends up being the case, the experiment design can be extended to examine other model architectures.

References

- Azarhooshang, N., Sengupta, P. & DasGupta, B. (2020), ‘A review of and some results for ollivier–ricci network curvature’, *Mathematics* **8**(9).
URL: <https://www.mdpi.com/2227-7390/8/9/1416>
- Forman, R. (2003), ‘Bochner’s method for cell complexes and combinatorial ricci curvature’, *Discrete & Computational Geometry* **29**, 323–374.
URL: <https://api.semanticscholar.org/CorpusID:9584267>
- Gomez, A. N., Zhang, I., Swersky, K., Gal, Y. & Hinton, G. E. (2018), ‘Targeted dropout’.
- Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. R. (2012), ‘Improving neural networks by preventing co-adaptation of feature detectors’.
URL: <https://arxiv.org/abs/1207.0580>
- Liu, Y., Zhou, C., Pan, S., Wu, J., Li, Z., Chen, H. & Zhang, P. (2023), Curvdrop: A ricci curvature based approach to prevent graph neural networks from over-smoothing and over-squashing, in ‘Proceedings of the ACM Web Conference 2023’, WWW ’23, Association for Computing Machinery, New York, NY, USA, p. 221–230.
URL: <https://doi.org/10.1145/3543507.3583269>
- Liu, Z., Xu, Z., Jin, J., Shen, Z. & Darrell, T. (2023), ‘Dropout reduces underfitting’.
URL: <https://arxiv.org/abs/2303.01500>
- Morerio, P., Cavazza, J., Volpi, R., Vidal, R. & Murino, V. (2017), ‘Curriculum dropout’.
URL: <https://arxiv.org/abs/1703.06229>
- Nguyen, K., Nong, H., Nguyen, V., Ho, N., Osher, S. & Nguyen, T. (2023), ‘Revisiting over-smoothing and over-squashing using ollivier-ricci curvature’.
URL: <https://arxiv.org/abs/2211.15779>
- Ni, C.-C., Lin, Y.-Y., Luo, F. & Gao, J. (2019), ‘Community detection on networks with ricci flow’, *Scientific reports* **9**(1), 1–12.
- Ollivier, Y. (2007), ‘Ricci curvature of markov chains on metric spaces’.
URL: <https://arxiv.org/abs/math/0701886>
- Samal, A., Sreejith, R. P., Gu, J., Liu, S., Saucan, E. & Jost, J. (2018), ‘Comparative analysis of two discretizations of ricci curvature for complex networks’, *Scientific Reports* **8**(1).
URL: <http://dx.doi.org/10.1038/s41598-018-27001-3>
- Saucan, E., Sreejith, R., Vivek-Ananth, R., Jost, J. & Samal, A. (2019), ‘Discrete ricci curvatures for directed networks’, *Chaos, Solitons and Fractals* **118**, 347–360.
URL: <http://dx.doi.org/10.1016/j.chaos.2018.11.031>
- Tan, S., Sia, J., Bogdan, P. & Ivanov, R. (2026), ‘Analyzing neural network information flow using differential geometry’.
URL: <https://arxiv.org/abs/2601.16366>
- Topping, J., Giovanni, F. D., Chamberlain, B. P., Dong, X. & Bronstein, M. M. (2022), Understanding over-squashing and bottlenecks on graphs via curvature, in ‘International Conference on Learning Representations’.
URL: <https://openreview.net/forum?id=7UmjRGzp-A>
- Weber, M., Saucan, E. & Jost, J. (2016), ‘Characterizing complex networks with forman-ricci curvature and associated geometric flows’.
URL: <https://arxiv.org/abs/1607.08654>